



# Puesta en producción

Del dominio recién comprado a Vistta funcionando en internet, en orden y con los comandos exactos. Ensayado de principio a fin; lo que no se ha podido probar está marcado.

---

## Antes de empezar

Lo que necesitas tener, y lo que puede esperar:

|                                       | Hace falta ya  | Se puede dejar para después              |
|---------------------------------------|----------------|------------------------------------------|
| Dominio                               | Sí             | —                                        |
| Acceso al DNS del dominio             | Sí             | —                                        |
| Un VPS con Docker                     | Sí             | —                                        |
| Cuenta de Cloudflare con tarjeta (R2) | No             | Sí: se estrena con el disco del servidor |
| Identidad fiscal para el aviso legal  | No para probar | Sí, antes de meter clientes reales       |
| Revisión jurídica de los textos       | No para probar | Sí, antes de meter clientes reales       |

**Puedes tenerlo funcionando hoy sin R2 y sin abogado.** Lo que no puedes es enseñárselo a un cliente de verdad sin las dos cosas, y eso está en el último apartado.

---

## La ruta, de un vistazo

| # | Paso                                            | Tiempo              | Bloquea a |
|---|-------------------------------------------------|---------------------|-----------|
| 1 | Apuntar el DNS                                  | 5 min + propagación | 5         |
| 2 | Preparar el servidor                            | 20 min              | 5         |
| 3 | Traer el código y escribir el <code>.env</code> | 15 min              | 5         |
| 4 | Elegir dónde van los medios                     | 5 min               | 5         |
| 5 | Construir y levantar                            | 15 min              | 6         |
| 6 | Crear el primer administrador                   | 5 min               | 7         |
| 7 | Comprobar que funciona                          | 10 min              | —         |
| 8 | Dejar las copias en marcha                      | 5 min               | —         |

**Haz el paso 1 lo primero** aunque no vayas a seguir hoy: la propagación del DNS tarda, y sin ella el certificado no se puede emitir.

---

---

## 1. Apuntar el DNS

En el panel de tu registrador, dos registros hacia la IP del servidor:

| Tipo | Nombre | Valor             | TTL |
|------|--------|-------------------|-----|
| A    | @      | <IP-DEL-SERVIDOR> | 300 |
| A    | www    | <IP-DEL-SERVIDOR> | 300 |

Un TTL bajo (300 s) mientras montas: si te equivocas, se corrige en cinco minutos en vez de en un día. Súbelo cuando esté estable.

Comprueba desde tu máquina antes de seguir:

```
dig +short TU-DOMINIO
```

Tiene que responder la IP del servidor. Si no responde, **espera**: Caddy pedirá el certificado y Let's Encrypt limita los intentos fallidos.

**No actives todavía el proxy naranja de Cloudflare** si usas su DNS. Ponlo en «solo DNS» hasta que todo funcione; activarlo obliga a cambiar dos cosas a la vez y está explicado en [DESPLIEGUE.md](#).

---

## 2. Preparar el servidor

Sobre Ubuntu 24.04, con 2 vCPU y 4 GB de RAM como punto de partida:

```
ssh root@<IP-DEL-SERVIDOR>

# Un usuario que no sea root para el día a día.
adduser vistta && usermod -aG sudo vistta

# Docker.
curl -fsSL https://get.docker.com | sh
usermod -aG docker vistta

# Cortafuegos: solo SSH y web.
ufw allow OpenSSH && ufw allow 80 && ufw allow 443 && ufw --force enable

# A partir de aquí, como `vistta`.
su - vistta
```

**Comprueba que Docker funciona sin sudo** antes de seguir:

```
docker run --rm hello-world
```

---

### 3. El código y el `.env`

```
git clone <URL-DEL-REPOSITORIO> /srv/vistta
cd /srv/vistta

cp deploy/env.produccion.ejemplo .env
chmod 600 .env
nano .env
```

Genera los secretos, **no los inventes**:

```
# Contraseña de la base
openssl rand -base64 32

# Clave de firma de los medios (32 caracteres o más)
docker run --rm node:22-alpine node -e "console.log(require('crypto').randomBytes(32).toString('base64'))"
```

Lo que no puede quedar vacío:

```
DOMINIO=tu-dominio.com
ACME_EMAIL=tu-correo@ejemplo.com
BASE_URL=https://tu-dominio.com
POSTGRES_PASSWORD=<lo que dio openssl>
MEDIA_SIGNING_KEY=<lo que dio el comando de arriba>
```

`MEDIA_SIGNING_KEY` **no se cambia después**. Cambiarla invalida todas las URL de medios ya emitidas, incluidos los pases abiertos en ese momento.

`BASE_URL` es lo que se pone delante de cada enlace de pase. Si está mal, los enlaces que generes apuntarán a otro sitio.

Las cuatro del aviso legal ( `TITULAR_NOMBRE` , `TITULAR_IDENTIFICACION` , `TITULAR_DIRECCION` , `CONTACTO_LEGAL` ) puedes dejarlas vacías para probar: la página `/legal` avisará en grande de que el despliegue no está configurado.

---

### 4. Dónde van los medios

Tres opciones. **Para estrenar, la primera.**

a) Disco del servidor — para probar hoy

```
STORAGE_DRIVER=fs
```

No hace falta contratar nada. Los medios van a un volumen de Docker y **sobreviven a redespigar** (probado). Dos límites que hay que conocer:

- **No entran en las copias de seguridad:** `backup.sh` vuelca la base, no estos bytes. Si se pierde el disco, se pierden las fotos.
- El tráfico de salida lo paga tu VPS.

Sirve para estrenar y para enseñárselo a alguien. **No para el trabajo real de un cliente.**

## b) Cloudflare R2 — el destino

```
STORAGE_DRIVER=r2
R2_ACCOUNT_ID=...
R2_ACCESS_KEY_ID=...
R2_SECRET_ACCESS_KEY=...
R2_BUCKET=vistta-medios
```

En el panel de Cloudflare: crea el bucket **privado y sin dominio público**, y un token de API de tipo *Object Read & Write* **limitado a ese bucket**.

**Aviso honesto: el adaptador de R2 nunca ha hablado con R2 de verdad.** Está verificado contra MinIO, que valida la firma igual, y por mutación. Pero el estreno contra R2 es el primer sitio donde puede aparecer una sorpresa, así que **hazlo con el paso 7 delante y sin clientes dentro**.

## c) Supabase Storage — el camino del MVP

```
STORAGE_DRIVER=supabase
SUPABASE_URL=...
SUPABASE_SECRET_KEY=...
```

Funciona y está probado, pero R2 es mejor destino: aquí cada visita relee el original para marcarlo, y Supabase cobra el tráfico de salida.

---

## 5. Construir y levantar

```
cd /srv/vistta
docker build -t vistta-api:latest .
docker build -f Dockerfile.web -t vistta-web:latest .

docker compose -f compose.prod.yml up -d
docker compose -f compose.prod.yml ps
```

Los cuatro servicios en `running`, y `api` y `db` en `healthy`. El servicio `migrar` aparece como `exited` y **eso es correcto**: corre las migraciones y termina.

Si `api` reinicia en bucle, el motivo está en el log y es legible:

```
docker compose -f compose.prod.yml logs api | tail -20
```

Caddy pide el certificado en el primer arranque. Tarda unos segundos:

```
docker compose -f compose.prod.yml logs caddy | grep -i certificate
```

---

## 6. El primer administrador

Desde la propia imagen, sin instalar Node en el servidor:

```
docker compose -f compose.prod.yml run --rm api node dist/crear-admin.js soporte "Soporte"
```

Enseña una contraseña temporal **una sola vez**. Apúntala y cámbiala al entrar.

Y la primera cuenta de cliente:

```
docker compose -f compose.prod.yml run --rm api \
  node dist/crear-usuario.js marina "Estudio Marina" "una-contrasena-larga"
```

**No hay ni habrá una ruta HTTP que conceda el rol de administrador.** Se da aquí, desde la máquina que tiene la base, y a propósito: un endpoint que otorgue admin convierte cualquier fallo de autorización futuro en una toma de control completa.

---

## 7. Comprobar que funciona

Esta secuencia es la que hay que ejecutar, en este orden. **Está probada tal cual**; solo cambia el dominio.

```
D=https://tu-dominio.com
```

```
# 1. La API responde
```

```
curl -s $D/health # {"ok":true}
```

```
# 2. El aviso legal está puesto (o dice que no)
```

```
curl -s $D/api/legal | grep -o '"completo":[a-z]*'
```

```
# 3. Cabeceras de seguridad
```

```
curl -sI $D/ | grep -i content-security-policy # script-src 'self'
```

```
curl -sI $D/ | grep -i x-robots-tag # noindex
```

```
# 4. Entrar y generar un pase
```

```
S=$(curl -s -X POST $D/api/panel/session -H 'content-type: application/json' \
-d '{"userId":"marina","password":"una-contrasena-larga"}' \
| python3 -c 'import sys,json;print(json.load(sys.stdin)["token"])')
```

```
P=$(curl -s -X POST $D/api/passes -H "authorization: Bearer $S" \
-H 'content-type: application/json' -d '{"profileId":"p_marina"}' \
| python3 -c 'import sys,json;print(json.load(sys.stdin)["url"])')
```

```
echo $P
```

```
# 5. EL INVARIANTE DEL PRODUCTO: se abre una vez y solo una
```

```
T=${P##*/v/}
```

```
curl -s -o /dev/null -w '%{http_code}\n' $D/api/open/$T # 200
```

```
curl -s -o /dev/null -w '%{http_code}\n' $D/api/open/$T # 410
```

Si el segundo no da 410, para y avisa. Es lo único que este producto promete de verdad.

### Y una comprobación que ninguna máquina hace por ti

Entra al panel con el navegador, sube una foto, genera un pase, ábrelo, y **mira que la foto lleva la marca de agua encima**. Es la mitad del producto y no hay comprobación automática que lo detecte en producción.

---

## 8. Dejar las copias en marcha

```
crontab -e
```

```
15 4 * * * cd /srv/vistta && ./scripts/backup.sh >> /var/log/vistta-backup.log 2>&1
```

Y **restaura una** antes de dar esto por hecho. El procedimiento está en `DESPLIEGUE.md`. Una copia que nunca se ha restaurado no es una copia, es un archivo.

---

---

## Cuando algo falla

| Síntoma                                   | Causa probable                            | Qué hacer                                                     |
|-------------------------------------------|-------------------------------------------|---------------------------------------------------------------|
| Caddy no consigue certificado             | El DNS no apunta aquí todavía             | <code>dig +short TU-DOMINIO</code> ; espera y reinicia Caddy  |
| <code>api</code> reinicia en bucle        | Configuración inválida o base inaccesible | <code>logs api</code> : imprime el motivo y sale con código 1 |
| Las subidas dan 500 con <code>fs</code>   | Permisos del volumen                      | Reconstruye la imagen: crea <code>/medios</code> con su dueño |
| Las subidas dan 413                       | Cuota del perfil agotada                  | Es el plan haciendo su trabajo                                |
| <code>/legal</code> dice «sin configurar» | Faltan las cuatro del titular             | Rellénalas y <code>up -d</code>                               |
| El pase abre siempre                      | <b>Grave.</b> Para y avisa                | —                                                             |
| Las fotos salen sin letras en la marca    | Faltan fuentes                            | No debería pasar: la construcción lo comprueba y falla        |

Los logs no llevan datos personales: registran método, **patrón** de ruta y tipo de error. La ruta real lleva el token del pase, que es una credencial.

---

## Antes de meter clientes de verdad

Nada de esto es programación, y sin ello **el despliegue no está listo para trabajo real de nadie**:

1. **Pasar a R2** y comprobar que sube, sirve y borra. Con `fs` , las fotos no están en ninguna copia.
2. **Rellenar el titular y el contacto legal.** Sin `CONTACTO_LEGAL` el procedimiento de retirada de contenido no lleva a ninguna parte.
3. **Fijar la jurisdicción** del VPS y del bucket, y anotarla en `legal/rat.md` .
4. **Guardar el contrato de encargado de cada proveedor.** Un subencargado sin contrato es un incumplimiento del art. 28.4, funcione el sistema como funcione.
5. **Decidir la retención del registro de acceso de Caddy**, que sí guarda IP y URL completa, y anotarla.
6. **Revisión de un abogado** de los cuatro textos públicos de `legal/` .
7. **Decidir los precios y** `GRACIA_CONGELADO_MS` , que hoy son provisionales. El segundo es el plazo tras el cual se destruye trabajo de un cliente.
8. **Restaurar una copia** de verdad.

---

## Actualizar, más adelante

```
cd /srv/vistta && git pull
docker build -t vistta-api:latest . && docker build -f Dockerfile.web -t vistta-web:latest .
docker compose -f compose.prod.yml up -d
```

`migrar` corre antes que `api` por dependencia declarada, así que las migraciones se aplican solas. Los certificados sobreviven porque están en un volumen.

---

## Qué se ha ensayado y qué no

|                                 |                                                                                                                                                                                                                                                          |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Ensayado entero en local</b> | Construcción de las dos imágenes, <code>compose up</code> , migraciones, alta de administrador y de cliente desde la imagen, la secuencia de comprobación del paso 7 (incluido el 200 → 410), subida con <code>fs</code> y persistencia tras redespargar |
| <b>Nunca ejecutado</b>          | Este mismo procedimiento <b>en un servidor real</b> , contra <b>R2 real</b> y con <b>un certificado de Let's Encrypt de verdad</b>                                                                                                                       |

Los tres primeros pasos son los que pueden dar una sorpresa. El resto está probado.